Practical No.: 10

Title: Demonstrating REST API in Node.js

Aim:

To create a **simple REST API** in **Node.js** using **Express.js** that performs basic CRUD operations.

Theory:

What is a REST API?

- **REST (Representational State Transfer)** is an architectural style for building **web services**.
- A **REST API** allows communication between a **client (frontend) and a server (backend)** using HTTP methods.

Common HTTP Methods in REST API:

Method	Description
GET	Retrieve data from the server
POST	Send data to the server
PUT	Update existing data
DELETE	Remove data from the server

Steps to Create a REST API in Node.js:

1. Install **Node.js** and **Express.js**.
2. Create an **Express server** and define REST API endpoints.
3. Test API using **Postman or a React frontend**.

Code Implementation:

Step 1: Create a Node.js Project and Install Dependencies

Open your terminal and run:

```
mkdir node-rest-api
cd node-rest-api
npm init -y
npm install express cors body-parser
```

Step 2: Create the REST API Server (`server.js`)

```
const express = require("express");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());
```

```
let students = [
    { id: 1, name: "Alice", course: "BCA" },
    { id: 2, name: "Bob", course: "MCA" }
];

// GET: Fetch all students
app.get("/students", (req, res) => {
    res.json(students);
});

// POST: Add a new student
app.post("/students", (req, res) => {
    const newStudent = { id: students.length + 1, ...req.body };
    students.push(newStudent);
    res.status(201).json(newStudent);
});

// PUT: Update student details
app.put("/students/:id", (req, res) => {
    const { id } = req.params;
    const index = students.findIndex(s => s.id == id);
    if (index !== -1) {
        students[index] = { ...students[index], ...req.body };
        res.json(students[index]);
    } else {
        res.status(404).send("Student not found");
    }
});

// DELETE: Remove a student
app.delete("/students/:id", (req, res) => {
    const { id } = req.params;
    students = students.filter(s => s.id != id);
    res.send("Student deleted");
});

app.listen(5000, () => console.log("Server running on port 5000"));
```

Step 3: Run the Server

node server.js

Step 4: Test the API

Method	Endpoint	Description	Example Request
GET	/students	Fetch all students	http://localhost:5000/students
POST	/students	Add new student	{ "name": "John", "course": "BCA" }
PUT	/students/1	Update student with ID 1	{ "course": "MBA" }
DELETE	/students/2	Delete student with ID 2	http://localhost:5000/students/2

Expected Output:

When calling **GET** `/students`, the response should be:

```
[
   { "id": 1, "name": "Alice", "course": "BCA" },
   { "id": 2, "name": "Bob", "course": "MCA" }
]
```

After **POSTing a new student**, the updated list will be:

```
[
   { "id": 1, "name": "Alice", "course": "BCA" },
   { "id": 2, "name": "Bob", "course": "MCA" },
   { "id": 3, "name": "John", "course": "BCA" }
]
```